

## Use cases

With serverless architecture, developers can deploy certain types of solutions at scale with cost-effectiveness. We have already discussed developing chatbots - it is a classic use case for serverless computing. Other key use cases for the serverless approach are given below.

- 1) **Three-tier Web applications:** Conventional single page applications (SPA), which rely on Representative State Transfer (REST) based services to perform a given functionality, can be re-written to leverage serverless functions front-ended by an API gateway. This is a powerful pattern that helps your application scale infinitely, without concerns of configuring scale-out or infrastructure resources..
- 2) **Scalable batch jobs:** Batch jobs were traditionally run as daemons or background processes on dedicated VMs. More often than not, this approach hit scalability and had reliability issues - developers would leave their critical processes with Single Points of Failure (SPoF). With the serverless approach, batch jobs can now be redesigned as a chain of mappers and reducers, each running as independent functions. Such mappers and reducers will share a common data store, something like a blob storage or a queue, and can individually scale up to meet the data processing needs.
- 3) **Stream processing:** Related to scalable batch jobs is the pattern of ingesting and processing large streams of data for near-real-time processing. Streams from services like Kafka and Kinesis can be processed by serverless functions, which can be scaled seamlessly to reduce latency and increase the throughput of the system. This pattern can elegantly handle spiky loads as well.
- 4) **Automation/event-driven processing:** Perhaps the first application of serverless computing was automation. Functions could be written to respond to certain alerts or events. These could also be periodically scheduled to augment the capabilities for the cloud service provider through extensibility.

The kind of applications that are best suited for serverless architectures include mobile back-ends, data processing systems (real-time and batch) and Web applications. In general, serverless architecture is suitable for any distributed system that reacts to events or process workloads dynamically, based on demand. For example, serverless computing is suitable for processing events from IoT (Internet of Things) devices, processing large data sets (in Big Data) and intelligent systems that respond to queries (chatbots).

## Serverless technologies

There are many proprietary and a few open source serverless technologies and platforms available for us to choose from.

AWS Lambda is the earliest (announced in late 2014 and released in 2015) and the most popular serverless technology, while other players are fast catching up. Microsoft's Azure

Functions has good support for a wider variety of languages and integrates with Microsoft's Azure services. Google's Cloud Functions is currently in beta. One of the key open source players in serverless technologies is Apache OpenWhisk, backed by IBM and Adobe. It is often tedious to develop applications directly on these platforms (AWS, Azure, Google and OpenWhisk). The serverless framework is a popular solution that aims to ease application development on these platforms.

Many solutions (especially open source) focus on abstracting away the details of container technologies like Docker and Kubernetes. Hyper.sh provides a container hosting service in which you can use Docker images directly in serverless style. Kubeless from Bitnami, Fission from Platform9, and funktion from Fabric8 are serverless frameworks that provide an abstraction over Kubernetes.

Given that serverless architecture is an emerging approach, technologies are still evolving and are yet to mature. So you will see a lot of action in this space in the years to come.

### Join us at the India Serverless Summit 2017

These are the best of times, and these are the worst of times! There are so many awesome new technologies to catch up on. But, we simply can't. We have seen a progression of computing models - from virtualisation, IaaS, PaaS, containers, and now, serverless - all in a matter of a few years. You certainly don't want to be left behind.

So join us at the Serverless Summit, India's first confluence on serverless technologies, being held on October 27, 2017 at Bengaluru. It is the best place to hear from industry experts, network with technology enthusiasts, as well as learn about how to adopt serverless architecture. The keynote speaker is John Willis, director of ecosystem development at Docker and a DevOps guru (widely known for the book 'The DevOps Handbook' that he co-authored).

*Open Source For You* is the media partner and the Cloud Native Computing Foundation is the community partner for this summit. For more details, please visit the website [www.inserverless.com](http://www.inserverless.com).

## Challenges in going serverless

Despite the fact that a few large businesses are already powered entirely by serverless technologies, we should keep in mind that serverless is an emerging approach. There are many challenges we need to deal with when developing serverless solutions. Let us discuss them in the context of the MovieBot example mentioned earlier.

### ▪ Debugging

Unlike in typical application development, there is no concept of a local environment for serverless functions. Even fundamental debugging operations like stepping-through, breakpoints, step-over and watch points are not available with serverless functions. As of now, we need to rely on extensive logging and instrumentation for debugging.